CLASE 3 · MÓDULO 2

# Práctica: TDD en TaskFlow

| Datos de la sesión | |
|---|---|
| **Duración** | 90 minutos |
| **Modalidad** | Equipos |
| **Herramientas** | Vitest, TypeScript, VS Code, repositorio TaskFlow |
| **Entregable** | Hito 1: repo con pipeline verde y tests corriendo |

## Contexto y objetivo

En el Módulo 1 vieron los fundamentos de Clean Code, SOLID y TDD. En este módulo aplican esos conceptos directamente sobre el código base de TaskFlow — el mismo proyecto que van a testear durante todo el semestre.

Al finalizar esta sesión cada equipo debe tener su repositorio listo para el Hito 1.

**Objetivos de la práctica**

- Ejecutar el ciclo Red-Green-Refactor al menos 3 veces sobre código real
- Escribir tests unitarios que corran sin base de datos (usando stubs)
- Detectar un bug intencional en el código base a través de los tests
- Dejar el repositorio del equipo con pipeline verde para el Hito 1

# 1. Setup del entorno

Antes de arrancar con los ejercicios, asegúrense de que el entorno funciona correctamente.

## 1.1 Clonar y configurar

| | |
|---|---|
| **1** | Hacer fork del repositorio base del curso en GitHub |
| **2** | Clonar el fork en su máquina local |
| **3** | Ejecutar el script de setup: bash setup.sh |
| **4** | Verificar que la DB esté corriendo (Docker o local) |
| **5** | Ejecutar: npm run test:unit desde la raíz del monorepo |

## 1.2 Verificar que los tests existentes pasan

Una vez que el entorno esté configurado, ejecuten los siguientes comandos y anoten el resultado:

| Comando | ¿Qué ejecuta? | ¿Pasó? ✓/✗ |
|---|---|---|
| npm run test:unit | Unit tests del backend (Vitest) | |
| npm run lint | ESLint + Prettier | |
| npm run typecheck | TypeScript sin errores de tipo | |

> ⚠ **Si algo falla en el setup**
> Antes de seguir, levanten la mano. No continúen con los ejercicios sin tener los tests existentes en verde — si la base está rota, no pueden distinguir si sus nuevos tests fallan por un bug o por un problema de configuración.

# 2. Estructura del código relevante

Para los ejercicios de hoy trabajan principalmente con estos archivos del backend:

| Archivo | Qué contiene |
|---|---|
| apps/api/src/services/task.service.ts | Lógica de negocio de tareas (validaciones, máquina de estados) |
| apps/api/src/services/auth.service.ts | Lógica de autenticación, registro, bloqueo de cuenta |
| apps/api/src/services/__tests__/task.service.test.ts | Tests unitarios existentes — referencia y punto de partida |
| apps/api/src/services/__tests__/auth.service.test.ts | Tests unitarios del servicio de auth |

## Dependencias inyectadas (DIP en acción)

Los servicios reciben sus dependencias por constructor, lo que permite inyectar stubs en los tests sin levantar la base de datos:

**Patrón de test con stub (referencia)**

```
// Ejemplo de cómo se crea un stub de repositorio en los tests existentes
const mockTaskRepo = {
  findById: vi.fn(),
  create:   vi.fn(),
  update:   vi.fn(),
};

const svc = new TaskService(mockTaskRepo as any);

// Luego en cada test:
mockTaskRepo.findById.mockResolvedValue(fakeTask);
const result = await svc.getTask('task-1');
expect(result).toEqual(fakeTask);
```

Lean los tests existentes en los archivos __tests__/ antes de escribir los propios. Son el mejor ejemplo del patrón que deben seguir.

# 3. Ejercicios TDD

Cada ejercicio presenta una especificación en lenguaje natural. Su trabajo es implementar los tests y el código usando el ciclo Red-Green-Refactor.

**Recordatorio: el ciclo TDD**

1. RED — Escribir el test que describe el comportamiento esperado. Ejecutar: debe fallar.
2. GREEN — Escribir el código mínimo para que el test pase. No más.
3. REFACTOR — Mejorar el código sin cambiar el comportamiento. Confirmar que los tests siguen en verde.

## EJERCICIO 1 Validaciones del título de una tarea

`Archivo: apps/api/src/services/task.service.ts`

Dificultad: ★☆☆ · Tiempo estimado: 20 minutos

### Especificación

El método validateTitle(title: string): void del TaskService debe cumplir las siguientes reglas de negocio:

| # | Comportamiento esperado | Input de ejemplo |
|---|---|---|
| 1 | Lanza error si el título tiene menos de 3 caracteres | "ab" → error |
| 2 | Lanza error si el título tiene más de 100 caracteres | "a".repeat(101) → error |
| 3 | Lanza error si el título está vacío o es solo espacios en blanco | "  " → error |
| 4 | Acepta títulos válidos sin lanzar error | "Mi tarea" → sin error |
| 5 | Acepta título con exactamente 3 caracteres (valor límite) | "abc" → sin error |
| 6 | Acepta título con exactamente 100 caracteres (valor límite) | "a".repeat(100) → sin error |

### Mensajes de error esperados

Los errores deben usar estos mensajes exactos (los tests van a verificar el mensaje):

- Título demasiado corto: "El título debe tener al menos 3 caracteres"
- Título demasiado largo: "El título no puede superar los 100 caracteres"
- Título vacío: "El título no puede estar vacío"

## Pista de implementación

Busquen si ya existe un método validateTitle en task.service.ts. Si existe, analicen qué casos faltan y agreguen los tests y la lógica que falta. Si no existe, créenlo desde cero con TDD.

## Criterio de aceptación

**El ejercicio está completo cuando:**

- Los 6 casos tienen su test escrito
- Todos los tests pasan con vitest run
- El código no usa any en la firma del método
- Los mensajes de error coinciden exactamente con los especificados

## EJERCICIO 2 Máquina de estados de una tarea

`Archivo: apps/api/src/services/task.service.ts`

Dificultad: ★★☆ · Tiempo estimado: 25 minutos

## Especificación

Las tareas en TaskFlow siguen una máquina de estados estricta:

**TODO → IN_PROGRESS → DONE**

Solo se permiten transiciones secuenciales. No se puede volver atrás ni saltear estados.

El método validateStatusTransition(currentStatus: TaskStatus, newStatus: TaskStatus): void debe cumplir:

| # | Comportamiento esperado | Ejemplo |
|---|---|---|
| 1 | Permite TODO → IN_PROGRESS | currentStatus=TODO, newStatus=IN_PROGRESS → sin error |
| 2 | Permite IN_PROGRESS → DONE | currentStatus=IN_PROGRESS, newStatus=DONE → sin error |
| 3 | Rechaza TODO → DONE (salto de estado) | currentStatus=TODO, newStatus=DONE → error |
| 4 | Rechaza IN_PROGRESS → TODO (retroceso) | currentStatus=IN_PROGRESS, newStatus=TODO → error |
| 5 | Rechaza DONE → cualquier estado (estado final) | currentStatus=DONE, newStatus=TODO → error |
| 6 | Rechaza transición al mismo estado | currentStatus=TODO, newStatus=TODO → error |

## Mensaje de error esperado

Todas las transiciones inválidas deben lanzar: "Transición de estado inválida: {currentStatus} → {newStatus}"

Ejemplo: "Transición de estado inválida: TODO → DONE"

## Criterio de aceptación

**El ejercicio está completo cuando:**

- Los 6 casos tienen su test escrito
- Todos los tests pasan con vitest run
- Se usa el enum TaskStatus (ya definido en el código base), no strings literales
- El mensaje de error incluye los valores reales de los estados

## EJERCICIO 3 Bloqueo de cuenta — encontrá el bug

`Archivo: apps/api/src/services/auth.service.ts`

Dificultad: ★★★ · Tiempo estimado: 25 minutos

🐛 **Este ejercicio contiene un bug intencional.**

Su trabajo es escribir los tests que especifican el comportamiento correcto. Cuando los ejecuten, uno de los tests va a fallar — eso significa que encontraron el bug. Luego deben corregirlo y confirmar que todos los tests pasan.

## Especificación

El sistema de autenticación debe bloquear una cuenta después de 5 intentos de login fallidos consecutivos. La lógica vive en el método handleFailedLogin(userId: string): Promise<void> del AuthService.

| # | Comportamiento esperado | Detalle |
|---|---|---|
| 1 | Después del 1er intento fallido: failedAttempts=1, cuenta NO bloqueada | isLocked debe ser false |
| 2 | Después del 4to intento fallido: failedAttempts=4, cuenta NO bloqueada | isLocked debe ser false |
| 3 | Después del 5to intento fallido: failedAttempts=5, cuenta SÍ bloqueada | isLocked debe ser true — este es el comportamiento correcto |
| 4 | Después del 6to intento fallido: cuenta ya bloqueada, no modifica failedAttempts | El conteo se detiene al bloquearse |

## Cómo escribir los tests

El AuthService recibe un repositorio de usuarios por constructor (DIP). Pueden mockearlo así:

```
const mockUser = { id: 'u1', email: 'test@test.com',
                   failedAttempts: N, isLocked: false };

const mockUserRepo = {
  findById: vi.fn().mockResolvedValue(mockUser),
  update:   vi.fn().mockImplementation(async (id, data) => {
    // Actualiza el mock para reflejar el update
    Object.assign(mockUser, data);
    return mockUser;
  })
};

const authSvc = new AuthService(mockUserRepo as any);
```

## Criterio de aceptación

**El ejercicio está completo cuando:**

- Los 4 casos tienen su test escrito
- Al ejecutar vitest run sin tocar el código, el test del caso 3 FALLA (confirmando el bug)
- Tras corregir el bug, TODOS los tests pasan
- Pueden explicar en una oración qué era el bug y cómo lo corrigieron

**Espacio para anotar el bug encontrado:**

La cuenta se bloqueaba con newFailedCount > 5. No se bloqueaba en el 5to intento, recién en el 6to.

*Descripción del fix aplicado:* Se crea una función nueva que lo contempla de manera correcta

# 4. Hito 1 — Configuración del repositorio del equipo

Una vez completados los ejercicios, configuren el repositorio de su equipo para el Hito 1. Este hito es el punto de partida de todo el trabajo acumulativo del semestre.

## Pasos del Hito 1

| # | Tarea | Responsable | ✓ |
|---|---|---|---|
| 1 | Crear organización o repositorio en GitHub con el nombre del equipo | | |
| 2 | Hacer fork del repositorio base del curso en ese repo | | |
| 3 | Agregar a todos los integrantes del equipo como colaboradores | | |
| 4 | Actualizar README.md: nombre del proyecto, integrantes, descripción breve | | |
| 5 | Verificar que el workflow de GitHub Actions (.github/workflows/) está presente | | |
| 6 | Hacer un push a main y confirmar que el pipeline queda en verde en GitHub | | |
| 7 | Agregar los tests del Ejercicio 1 al repo (al menos 3 tests pasando) | | |
| 8 | Confirmar: vitest run sin errores en la rama main | | |

## Criterio mínimo de aprobación del Hito 1

- ESLint sin errores (npm run lint pasa en CI)
- TypeScript sin errores de tipo (npm run typecheck pasa en CI)
- Al menos 3 unit tests nuevos pasando (los del Ejercicio 1)
- Pipeline de GitHub Actions en verde en la rama main
- README.md actualizado con datos del equipo

**URL del repositorio del equipo:**

_____

Estado del pipeline al final de la clase: ☐ Verde ☐ En progreso ☐ Rojo (con issue abierto)

# 5. Referencia rápida

## Comandos útiles

| Comando | Qué hace |
|---|---|
| npm run test:unit | Ejecuta todos los unit tests con Vitest |
| npm run test:unit -- --watch | Modo watch: re-ejecuta al guardar |
| npm run test:unit -- --coverage | Ejecuta con reporte de cobertura |
| npm run test:unit -- task.service | Filtra y ejecuta solo tests de task.service |
| npm run lint | Ejecuta ESLint + Prettier check |
| npm run lint:fix | Corrige automáticamente los errores de formato |
| npm run typecheck | Verifica tipos TypeScript sin compilar |

## Matchers de Vitest más usados en esta práctica

| Matcher | Cuándo usarlo |
|---|---|
| expect(fn).toThrow('mensaje') | Verificar que se lanza un error con mensaje exacto |
| expect(fn).not.toThrow() | Verificar que NO se lanza error |
| expect(value).toBe(expected) | Comparación estricta (===) |
| expect(value).toBeTruthy() / toBeFalsy() | Verificar verdadero/falso |
| expect(mockFn).toHaveBeenCalledWith(args) | Verificar que el mock fue llamado con ciertos args |
| vi.fn().mockResolvedValue(val) | Stub que resuelve una promesa con el valor dado |
| vi.fn().mockRejectedValue(err) | Stub que rechaza una promesa con el error dado |

## Estructura de un test bien escrito

```
describe('TaskService.validateTitle', () => {

  it('debe rechazar títulos menores a 3 caracteres', () => {
    // Arrange
    const svc = new TaskService(mockRepo as any);

    // Act & Assert
    expect(() => svc.validateTitle('ab'))
      .toThrow('El título debe tener al menos 3 caracteres');
```

```
    });

    it('debe aceptar un título válido', () => {
      const svc = new TaskService(mockRepo as any);
      expect(() => svc.validateTitle('Tarea válida')).not.toThrow();
    });

  });
```